



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS BLUMENAU
ENGENHARIA DE CONTROLE E AUTOMAÇÃO

Thiago Jaunatre Faquinello, Sione Serrano Pedro

**Desenvolvimento de um Sistema de Controle para Elevador de Duas Cabines
Utilizando C/C++ e Assembly MIPS**

BLUMENAU
2025

LISTA DE FIGURAS

FIGURA 1 – Montagem Tinkercad.....	9
------------------------------------	---

LISTA DE TABELAS

Tabela 1 - Funcionamento do servomotor.....	9
Tabela 2 – Descrição dos estados.....	10
Tabela 3 – Mapeamento das teclas para Assembly MIPS.....	18

SUMÁRIO

1	IDENTIFICAÇÃO DA PROPOSTA.....	5
1.1	IDENTIFICAÇÃO DOS PARTICIPANTES	5
2	INTRODUÇÃO.....	6
2.1	OBJETIVOS GERAIS	7
2.1.1	Objetivos específicos	7
3	DESENVOLVIMENTO	8
3.1	DESCRIÇÃO DO PROJETO.....	8
3.2	COMPONENTES UTILIZADOS	8
3.3	MONTAGEM DA SIMULAÇÃO	9
3.4	LÓGICA DE FUNCIONAMENTO	10
3.5	DISTRIBUIÇÃO DAS CHAMADAS DE CORREDOR	10
3.6	IMPLEMENTAÇÃO EM C/C++	11
3.7	TRADUÇÃO PARA ASSEMBLY MIPS	18
3.8	RESULTADOS OBTIDOS.....	18
4	CONCLUSÕES.....	20
	REFERÊNCIAS	21

1 IDENTIFICAÇÃO DA PROPOSTA

Título: Desenvolvimento de um Sistema de Controle para Elevador de Duas Cabines Utilizando C/C++ e Assembly MIPS.

Tema principal: Desenvolvimento de Sistemas de Controle utilizando Microprocessadores.

Proposta: Implementar a lógica de funcionamento de um elevador de duas cabines com quatro andares, desenvolver inicialmente em C/C++, validar através de simulação no Tinkercad e posteriormente converter o algoritmo para Assembly MIPS.

Período para execução e finalização do projeto: Projeto iniciado em 2 de junho de 2026 e finalizado em 23 de junho de 2026.

1.1 IDENTIFICAÇÃO DOS PARTICIPANTES

Participante 1: Thiago Jaunatre Faquinello. Responsável pela implementação da lógica de controle do elevador em linguagem C/C++ e documentação do projeto. Possui conhecimentos em programação, algoritmos, microcontroladores e arquitetura de computadores.

Participante 2: Sione Serrano Pedro. Responsável pela tradução do código para Assembly MIPS, validação dos resultados e documentação do projeto. Possui conhecimentos em arquitetura de computadores, programação e microcontroladores.

2 INTRODUÇÃO

A automação de sistemas de transporte vertical, como elevadores, é um campo de estudo que combina conceitos de engenharia de controle, programação de sistemas embarcados e arquitetura de computadores. A eficiência desses sistemas depende diretamente da lógica de controle empregada, que deve ser capaz de gerenciar múltiplas solicitações simultâneas, otimizar o tempo de resposta e garantir o correto funcionamento do sistema. Nesse contexto, a compreensão das diferentes camadas de abstração na programação — desde linguagens de alto nível, como C++, até linguagens de baixo nível, como Assembly MIPS — torna-se fundamental para o desenvolvimento de controladores eficientes e para a formação de engenheiros capacitados a compreender a interação entre hardware e software.

Este relatório apresenta o desenvolvimento de um sistema de controle para um elevador de duas cabines e quatro andares, utilizando como plataforma de prototipagem o Arduino Uno. O trabalho contempla a implementação inicial da lógica de funcionamento em linguagem C++, seguida da validação do sistema por meio de simulação no ambiente Tinkercad. Posteriormente, o algoritmo foi traduzido para a linguagem Assembly MIPS, permitindo analisar a implementação da mesma lógica de controle em um nível mais próximo da arquitetura do processador.

A relevância deste estudo reside na integração de conhecimentos teóricos e práticos relacionados a microprocessadores, microcontroladores e sistemas embarcados. Além disso, a adoção de uma abordagem que contempla tanto a simulação quanto a implementação em diferentes níveis de abstração permite analisar as características de cada linguagem e compreender como algoritmos de controle podem ser representados desde linguagens de alto nível até instruções diretamente executadas pelo processador.

Para adequação à proposta do projeto, foi adotado um sistema de chamadas externas baseado na seleção automática da cabine mais próxima do andar solicitado. Em situações de igualdade de distância entre as cabines, foi implementado um critério de desempate baseado no estado operacional do sistema, garantindo a continuidade do atendimento das solicitações.

2.1 OBJETIVOS GERAIS

Desenvolver e validar um sistema de controle para um elevador de duas cabines utilizando programação em C/C++ e Assembly MIPS.

2.1.1 Objetivos específicos

- Implementar a lógica do elevador em C/C++;
- Simular o sistema no Tinkercad;
- Testar diferentes cenários de chamadas;
- Traduzir o código para Assembly MIPS;
- Documentar o desenvolvimento do projeto.

3 DESENVOLVIMENTO

3.1 DESCRIÇÃO DO PROJETO

O projeto consiste na implementação de um sistema de controle para um elevador de duas cabines e quatro andares utilizando um Arduino Uno. O objetivo foi desenvolver inicialmente a lógica de funcionamento em linguagem C/C++, mostrar seu funcionamento pro meio da simulação no Tinkercad e posteriormente traduzir o algoritmo para MIPS.

Cada cabine possui quatro botões internos correspondentes aos quatro andares disponíveis. Além disso, cada andar possui um botão externo localizado nos corredores, responsáveis por solicitar uma cabine.

O sistema foi projetado para atender as chamadas respeitando as restrições dadas na proposta do projeto, que analisa a direção atual e deslocamento das cabines. Dessa forma, chamadas incompatíveis com o sentido atual de movimento são armazenadas e atendidas posteriormente.

3.2 COMPONENTES UTILIZADOS

Para a implementação da simulação foram utilizados os seguintes componentes:

- 1 Arduino Uno;
- 2 servomotores;
- 12 botões;
- 6 protoboards;
- Fios de conexão.

Os servomotores foram utilizados para representar o deslocamento das cabines (para realizar o controle do componente, foram utilizados os códigos de exemplo descritos no tutorial (GUSE, 2024).). Cada posição angular corresponde a um andar.

Tabela 1 - Funcionamento do servomotor

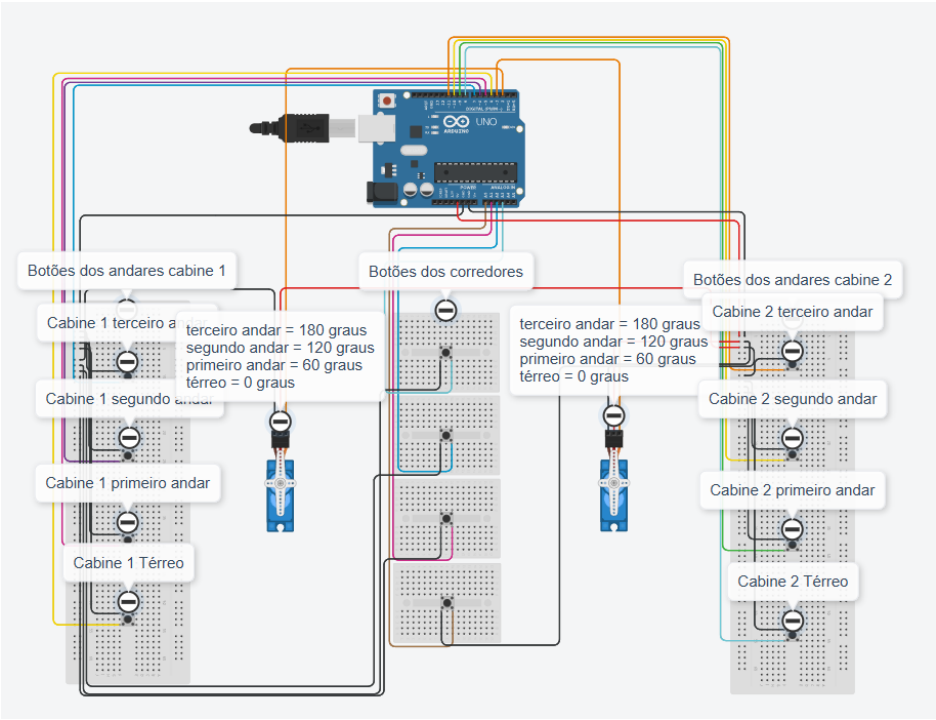
Andar	Ângulo
Térreo	0°
1° andar	60°
2° andar	120°
3° andar	180°

Fonte: elaborado pelos autores.

3.3 MONTAGEM DA SIMULAÇÃO

A figura 1 apresenta a montagem desenvolvida no Tinkercad.

Figura 1 – Montagem Tinkercad



Fonte: elaborado pelos autores.

A estrutura foi dividida em três grupos principais de entradas:

- Botões internos da Cabine 1;
- Botões de chamada externa dos corredores;
- Botões internos da Cabine 2.

Os servomotores representam a posição das cabines e movimentam-se entre os ângulos correspondentes aos andares.

A simulação está disponível em:

<https://www.tinkercad.com/things/kHrCkrihPRB-trab-final-lab/editel?returnTo=https%3A%2F%2Fwww.tinkercad.com%2Fdashboard>

3.4 LÓGICA DE FUNCIONAMENTO

O sistema utiliza uma estratégia baseada em filas de chamadas para cada cabine.

Cada elevador mantém:

- Andar atual;
- Direção atual;
- Estado de operação;
- Lista de chamadas pendentes.

Os estados possíveis são:

Tabela 2 – Descrição dos estados

Estado	Descrição
0	Parado
1	Em movimento
2	Porta aberta

Fonte: elaborado pelos autores.

Quando uma nova chamada é registrada, ela é armazenada na fila da cabine correspondente e processado conforme a direção atual de deslocamento, seguindo as restrições impostas na proposta do projeto.

3.5 DISTRIBUIÇÃO DAS CHAMADAS DE CORREDOR

As chamadas externas são atribuídas automaticamente à cabine mais próxima.

Para realizar essa decisão, o sistema calcula a diferença entre o ângulo atual de cada servomotor e o ângulo correspondente ao andar solicitado.

A cabine que tiver a menor distância angular recebe a chamada.

A distância é calculada pela expressão:

$$Distância = |\hat{ânguloAtual} - \hat{ânguloDestino}|$$

Critério de Desempate

Caso as duas cabines estejam exatamente à mesma distância do andar solicitado, o sistema utiliza o seguinte critério:

- Se a Cabine 1 estiver parada, a chamada é atribuída à Cabine 1;
- Caso contrário, a chamada é atribuída à Cabine 2.

Dessa forma, esse processo evita ambiguidades durante a distribuição automática das chamadas.

3.6 IMPLEMENTAÇÃO EM C/C++

```
//Responsável pelo controle dos servomotores que representam as cabines.
#include <Servo.h>

Servo servo1;
Servo servo2;
//Define os pinos utilizados pelos servomotores.
const int pinoServo1 = 2;
const int pinoServo2 = 3;
//Armazenam os pinos correspondentes aos botões de cada cabine e dos
corredores.
const int botoesCabine1[4] = {4, 5, 6, 7};
const int botoesCabine2[4] = {8, 9, 10, 11};
const int botoesCorredor[4] = {A0, A1, A2, A3};
//Relaciona cada andar ao ângulo correspondente do servomotor.
const int angulosAndar[4] = {0, 60, 120, 180};

bool chamadas1[4] = {false, false, false, false};
int andarAtual1 = 0;
int direcao1 = 0;
```

```

int anguloServo1 = 0;
int estado1 = 0;
unsigned long portaTimer1 = 0;
unsigned long servoTimer1 = 0;

bool chamadas2[4] = {false, false, false, false};
int andarAtual2 = 0;
int direcao2 = 0;
int anguloServo2 = 0;
int estado2 = 0;
unsigned long portaTimer2 = 0;
unsigned long servoTimer2 = 0;

// A função setup() inicializa os servomotores, configura os pinos de entrada e
posiciona as cabines no térreo.
void setup() {
    Serial.begin(9600);
    servo1.attach(pinoServo1);
    servo2.attach(pinoServo2);

    for(int i = 0; i < 4; i++) {
        pinMode(botoesCabine1[i], INPUT_PULLUP);
        pinMode(botoesCabine2[i], INPUT_PULLUP);
        pinMode(botoesCorredor[i], INPUT_PULLUP);
    }

    servo1.write(anguloServo1);
    servo2.write(anguloServo2);
    Serial.println("Sistema de Elevadores Duplos (Arduino UNO) Iniciado.");
}

//Verifica se existe alguma solicitação pendente acima da posição atual da
Cabine 1.
bool temChamadaAcima1() { for(int i=andarAtual1+1; i<4; i++)
if(chamadas1[i]) return true; return false; }

```

//Verifica se existe alguma solicitação pendente abaixo da posição atual da Cabine 1.

```
bool temChamadaAbaixo1() { for(int i=0; i<andarAtual1; i++) if(chamadas1[i])  
return true; return false; }
```

//Verifica se existe alguma solicitação pendente acima da posição atual da Cabine 2.

```
bool temChamadaAcima2() { for(int i=andarAtual2+1; i<4; i++)  
if(chamadas2[i]) return true; return false; }
```

//Verifica se existe alguma solicitação pendente abaixo da posição atual da Cabine 2.

```
bool temChamadaAbaixo2() { for(int i=0; i<andarAtual2; i++) if(chamadas2[i])  
return true; return false; }
```

// A função lerBotoes() realiza a leitura de todos os botões do sistema, registra novas solicitações e distribui as chamadas de corredor para a cabine mais próxima.

```
void lerBotoes() {  
for(int i = 0; i < 4; i++) {  
if(digitalRead(botoesCabine1[i]) == LOW && !chamadas1[i]) {  
chamadas1[i] = true;  
Serial.print("Cabine 1: Destino adicionado -> Andar "); Serial.println(i + 1);  
}  
}
```

```
for(int i = 0; i < 4; i++) {  
if(digitalRead(botoesCabine2[i]) == LOW && !chamadas2[i]) {  
chamadas2[i] = true;  
Serial.print("Cabine 2: Destino adicionado -> Andar "); Serial.println(i + 1);  
}  
}
```

```
for(int i = 0; i < 4; i++) {  
if(digitalRead(botoesCorredor[i]) == LOW) {  
if(!chamadas1[i] && !chamadas2[i] && !(andarAtual1 == i && estado1 ==  
2) && !(andarAtual2 == i && estado2 == 2)) {
```

```

int anguloAlvo = angulosAndar[i];

int dist1 = abs(anguloServo1 - anguloAlvo);
int dist2 = abs(anguloServo2 - anguloAlvo);

if(dist1 < dist2) {
    chamadas1[i] = true;
    Serial.print("Corredor: Chamada no "); Serial.print(i+1);
    Serial.print("o Andar enviada para Cabine 1 (Mais Perto por ");
    Serial.print(dist2 - dist1); Serial.println(" graus de vantagem");
}
else if(dist2 < dist1) {
    chamadas2[i] = true;
    Serial.print("Corredor: Chamada no "); Serial.print(i+1);
    Serial.print("o Andar enviada para Cabine 2 (Mais Perto por ");
    Serial.print(dist1 - dist2); Serial.println(" graus de vantagem");
}
else {
    if(direcao1 == 0) {
        chamadas1[i] = true;
        Serial.println("Corredor: Empate absoluto! Enviado para Cabine 1");
    } else {
        chamadas2[i] = true;
        Serial.println("Corredor: Empate absoluto! Enviado para Cabine 2");
    }
}
delay(200);
}
}
}
}

```

// A função loop() executa continuamente a lógica principal do sistema, controlando movimentação, abertura e fechamento das portas e atualização dos estados das cabines.

```
void loop() {  
  lerBotoes();  
  unsigned long tempoAtual = millis();  
  
  if (estado1 == 0) {  
    if (chamadas1[andarAtual1]) {  
      estado1 = 2;  
      portaTimer1 = tempoAtual;  
      chamadas1[andarAtual1] = false;  
      Serial.print("Cabine 1: Porta Aberta no "); Serial.print(andarAtual1 + 1);  
      Serial.println("o Andar.");  
    } else if (temChamadaAcima1()) {  
      direcao1 = 1; estado1 = 1; servoTimer1 = tempoAtual;  
    } else if (temChamadaAbaixo1()) {  
      direcao1 = -1; estado1 = 1; servoTimer1 = tempoAtual;  
    }  
  }  
  else if (estado1 == 1) {  
    if (tempoAtual - servoTimer1 >= 30) {  
      servoTimer1 = tempoAtual;  
      int alvoAngulo = angulosAndar[andarAtual1 + direcao1];  
  
      if (anguloServo1 < alvoAngulo) anguloServo1++;  
      else if (anguloServo1 > alvoAngulo) anguloServo1--;  
      servo1.write(anguloServo1);  
  
      if (anguloServo1 == alvoAngulo) {  
        andarAtual1 += direcao1;
```

```

        if (chamadas1[andarAtual1] || (direcao1 == 1 && !temChamadaAcima1())
|| (direcao1 == -1 && !temChamadaAbaixo1())) {
            estado1 = 2;
            portaTimer1 = tempoAtual;
            chamadas1[andarAtual1] = false;
            Serial.print("Cabine 1: Parando no "); Serial.print(andarAtual1 + 1);
Serial.println("o Andar.");
        }
    }
}
else if (estado1 == 2) {
    if (tempoAtual - portaTimer1 >= 5000) {
        estado1 = 0; direcao1 = 0;
        Serial.print("Cabine 1: Porta Fechada no "); Serial.print(andarAtual1 + 1);
Serial.println("o Andar.");
    }
}

```

```

if (estado2 == 0) {
    if (chamadas2[andarAtual2]) {
        estado2 = 2;
        portaTimer2 = tempoAtual;
        chamadas2[andarAtual2] = false;
        Serial.print("Cabine 2: Porta Aberta no "); Serial.print(andarAtual2 + 1);
Serial.println("o Andar.");
    } else if (temChamadaAcima2()) {
        direcao2 = 1; estado2 = 1; servoTimer2 = tempoAtual;
    } else if (temChamadaAbaixo2()) {
        direcao2 = -1; estado2 = 1; servoTimer2 = tempoAtual;
    }
}

```



```

else if (estado2 == 1) {
    if (tempoAtual - servoTimer2 >= 30) {
        servoTimer2 = tempoAtual;
        int alvoAngulo = angulosAndar[andarAtual2 + direcao2];

        if (anguloServo2 < alvoAngulo) anguloServo2++;
        else if (anguloServo2 > alvoAngulo) anguloServo2--;
        servo2.write(anguloServo2);

        if (anguloServo2 == alvoAngulo) {
            andarAtual2 += direcao2;

            if (chamadas2[andarAtual2] || (direcao2 == 1 && !itemChamadaAcima2())
|| (direcao2 == -1 && !itemChamadaAbaixo2())) {
                estado2 = 2;
                portaTimer2 = tempoAtual;
                chamadas2[andarAtual2] = false;
                Serial.print("Cabine 2: Parando no "); Serial.print(andarAtual2 + 1);
Serial.println("o Andar.");
            }
        }
    }
}
else if (estado2 == 2) {
    if (tempoAtual - portaTimer2 >= 5000) {
        estado2 = 0; direcao2 = 0;
        Serial.print("Cabine 2: Porta Fechada no "); Serial.print(andarAtual2 + 1);
Serial.println("o Andar.");
    }
}
}

```

3.7 TRADUÇÃO PARA ASSEMBLY MIPS

Tabela 3 – Mapeamento das teclas para Assembly MIPS

Tecla	Função
1,2,3,4	Chamada do corredor nos andares 1-4
Q,W,E,R	Botão da cabine 1 (andares1-4)
A,S,D,F	Botão da cabine 2 (andares1-4)

Fonte: elaborado pelos autores.

Link de acesso para o arquivo ASM com o código de MIPS (o código está comentado explicando cada função do mesmo modo que foi realizado no código em C/C++):

https://github.com/thiagofaquinello/Microprocessadores-relatorio/blob/main/codigo_mips.asm

Ao colocar esse código no MARS 4.5, para poder realizar a simulação desse código utilizando o mapeamento de chaves apresentado anteriormente, deve-se seguir um passo-a-passo:

1. No menu superior do MARS, vá em **Tools**(Ferramentas);
2. Selecione **Keyboard and Display MMIO Simulator**;
3. Clique no botão **Connect to MIPS** na janelinha que abriu;
4. Digite as teclas de comando (como 1, w, f) na caixa de texto da parte inferior dessa ferramenta para registrar as chamadas nos elevadores.

O MARS não pode ser usado diretamente no serial monitor proque o código usa uma técnica chamada MMIO (Entrada/Saída Mapeada em Memória), fazendo com que ele “vigie” diretamente o endereço físico de memória para checar se alguma tecla foi pressionada.

Portanto, é graças a essa janelinha de MMIO que o código é não-bloqueante, ou seja, o elevador consegue continuar subindo e descendo de forma fluida enquanto o programa, ao mesmo tempo, fica livre para notar se alguma tecla foi pressionada no meio do caminho.

3.8 RESULTADOS OBTIDOS

Os testes realizados demonstram na simulação demonstram que:

- As duas cabines operam de forma independente;
- As chamadas internas são corretamente registradas;
- AS chamadas externas são encaminhadas para a cabine mais próxima seguindo as restrições impostas pelo enunciado do projeto;
- O critério de desempate funciona corretamente;
- As cabines respeitam o sentido atual de deslocamento;
- As portas permanecem abertas durante o intervalo programado;
- A implementação em Assembly reproduz o comportamento da versão desenvolvida em C/C++, considerando as limitações da linguagem.

4 CONCLUSÕES

O presente trabalho alcançou os objetivos propostos, consolidando o desenvolvimento de um sistema de controle para elevador de duas cabines capaz de operar de forma autônoma e atender às solicitações dos usuários conforme as regras definidas para o projeto. A implementação em linguagem C++ mostrou-se adequada para a modelagem da lógica de funcionamento, permitindo a construção de um código organizado e modular, com gerenciamento independente das chamadas de cada cabine, controle de direção e gerenciamento dos estados operacionais (parado, em movimento e porta aberta).

Os testes realizados no ambiente de simulação Tinkercad demonstraram o correto funcionamento do sistema, evidenciando que as chamadas internas são registradas adequadamente e que as solicitações externas são distribuídas para a cabine mais próxima por meio do critério de distância angular adotado. Em situações de igualdade de distância entre as cabines, foi utilizado um critério de desempate baseado no estado da Cabine 1, priorizando-a quando se encontra parada e encaminhando a solicitação para a Cabine 2 nos demais casos. Além disso, as restrições relacionadas ao sentido de deslocamento foram respeitadas e o tempo de abertura das portas foi controlado conforme especificado no projeto.

A tradução do algoritmo para Assembly MIPS representou uma etapa de maior complexidade, exigindo a compreensão detalhada da arquitetura do processador, do gerenciamento de memória e das instruções de controle de fluxo. A utilização do simulador MARS permitiu validar a implementação desenvolvida em baixo nível, demonstrando que os testes realizados produziram comportamento compatível com a lógica originalmente implementada em C++.

O desenvolvimento do projeto possibilitou aplicar na prática conceitos estudados ao longo da disciplina de Microprocessadores, especialmente aqueles relacionados à programação em C++, programação em Assembly MIPS, controle lógico de sistemas e interação entre hardware e software. Dessa forma, o trabalho contribuiu para uma compreensão mais aprofundada dos diferentes níveis de abstração envolvidos no desenvolvimento de sistemas computacionais e embarcados.

REFERÊNCIAS

GUSE, Rosana. ***Controlando um servo motor com Arduino: 6 modos diferentes.*** MakerHero, 2024. Disponível em: <https://www.makerhero.com/blog/controlando-um-servo-motor-com-arduino/>. Acesso em: 2 jun. 2026.